

dataroots

DevOps



Course overview

1. What is DevOps
2. CI/CD
3. CI/CD Extra topics
4. CI/CD Demo
5. CI/CD Exercises

What is DevOps

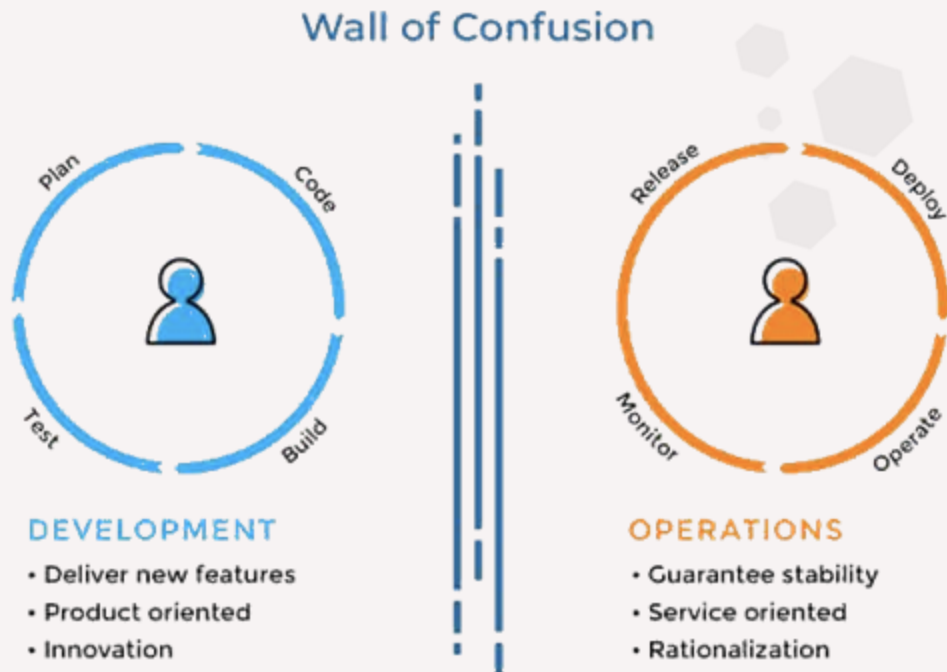


What do you think DevOps is?

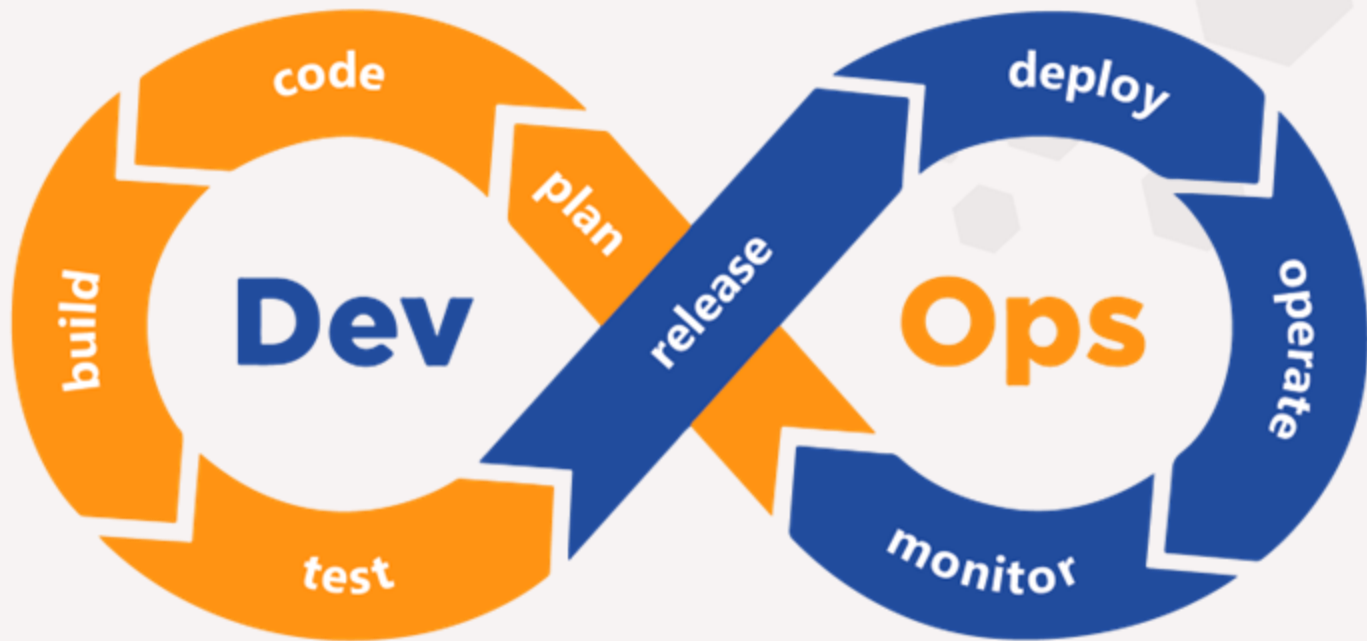
What is DevOps?

*“A set of **practices** intended to reduce the time between committing a **change** to a system and the change being placed into normal production, while ensuring **high quality**”*

How we used to do things



What DevOps solves



dataroots

Source: <https://www.delta-n.nl/devops/wat-is-devops/>

What is DevOps not?

- It's not about the tools
- It's not a role
- **It's a mindset**

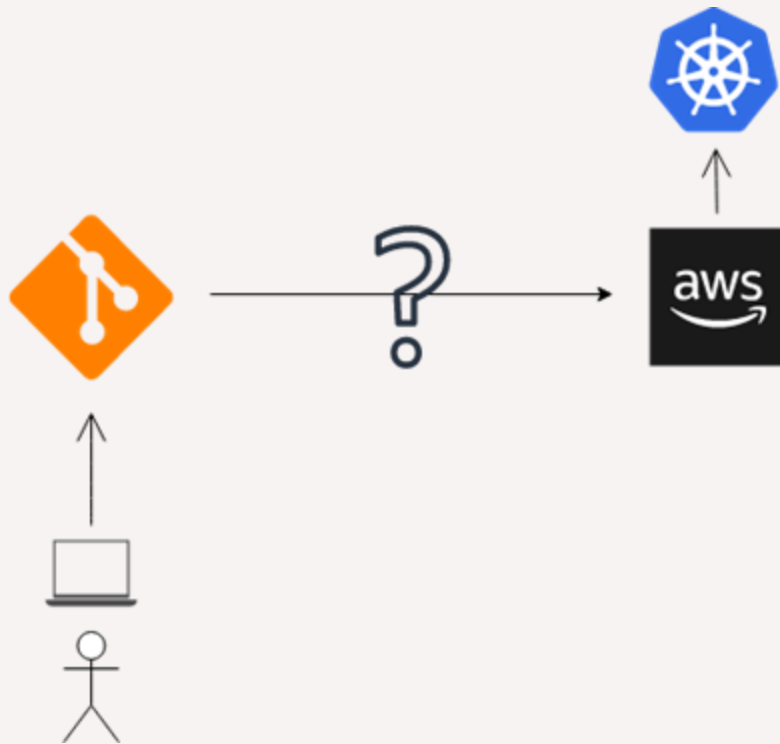


end-to-end responsibility

CI/CD

What you've seen so far

- git
- Python
 - pytest
 - flake
 - mypy
 - Mkdocs
- Containers/docker
- Cloud (AWS/Azure/GCP)
- Terraform
- K8s
- bash
- ...





git



How do we link these things?



That's something we will see today

What is CI/CD?

What is CI/CD?

Wikipedia definition:

CI/CD **bridges** the gaps between **development** and **operation** activities by enforcing **automation**.



Why is CI/CD important?

- It defines how your **team collaborates**
- It defines how your **software/application** gets **released**
 - How you go from the laptop on your code to the release to the cloud
- Release code **changes** more **frequently** and **reliably**
- You set it up once, and it **automatically** checks/does things for you

=> Don't do this or get this wrong and you can **lose** a lot of **time/money**

Automation

- Most important part of CI/CD
- We hate doing things manually
- Done the same way every time / repeatable
- Scripting, usually done in:
 - bash/sh
 - python



Automation

N

most important thing of cica. in one word



Automation.

CI

Continuous Integration

What?

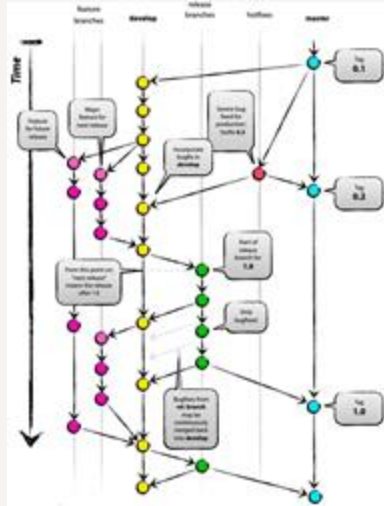
- How does a team **collaborate**?
- How do we keep **quality** of our **code** high?

How?

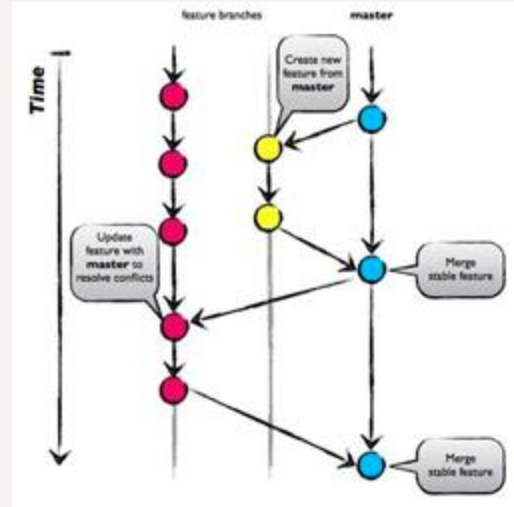
- **Branching/Merging strategies** (GitFlow, Gitlab Flow, Trunk-based)
- **Automated** linting, testing, test cov, security/code smell checks, build artifacts, ...
- Repository **policies** (Always go via Pull Request)

Makes sure new commits can be added easily and don't break the code

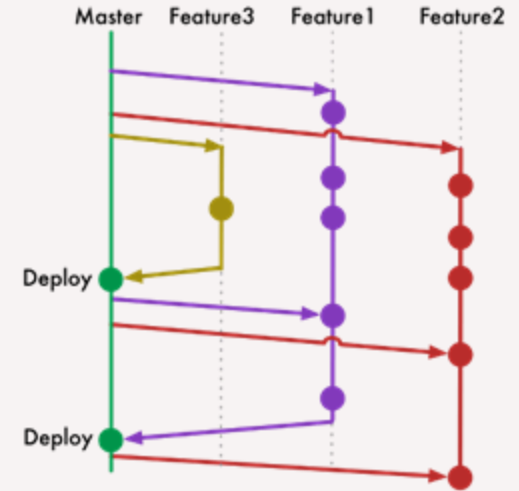
CI - Branching strategies



GitFlow



GitHub Flow



Trunk based

Every team/client does it slightly different, but try keep it as simple as possible

*there are more strategies than shown here

CD

Continuous **D**elivery or Continuous **D**eployment

What?

- When there are code changes, deploy to staging environment
- When ready to release and staging is ok, deploy to production

How?

- **Automate** deployment to different environments

High level, how does a CI/CD system work (1)

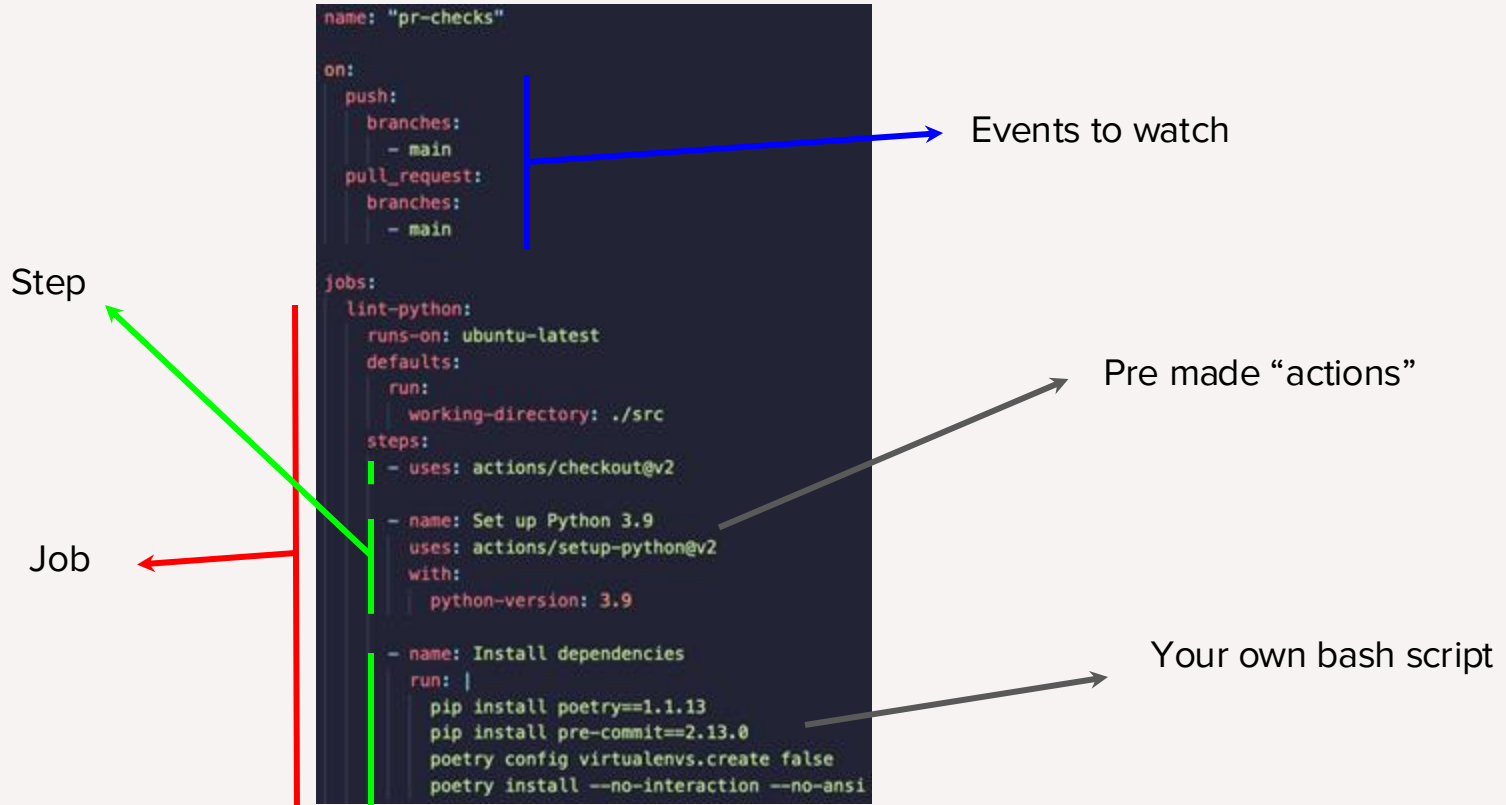
2 Parts (can be a separate tool or the same tool)

- Your version control system (**VCS**) (Github, Bitbucket, Gitlab, Azure Devops, ...)
 - Where you store the code
 - Your build system (**BS**) (Github, Gitlab, Jenkins, Azure Devops, Circle CI, ...)
 - Where you automate stuff
1. You define your pipeline via some config file (for example YAML)
 2. BS watches for events (push, merge, tag, ...) coming from VCS
 3. When BS sees event from VCS, BS triggers the correct action(s)

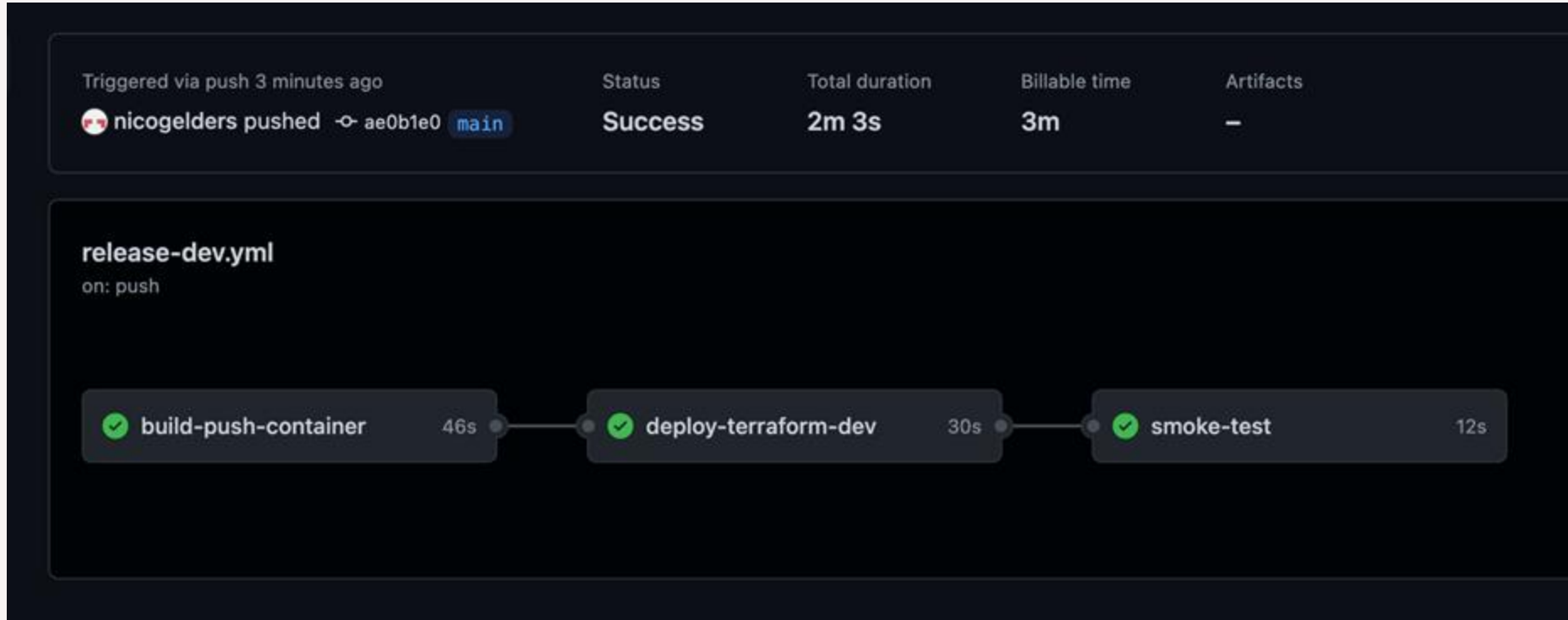
All VCS/Build tools do pretty much the same (some have there small quirks)

Which one you will use will depend on the client you work for.

High level, how does CI/CD work (2)



High level, how does a CI/CD system work (3)



CI/CD Extra topics

DevOps extensions

- GitOps

- Focuses on deploying IaC in a continuous manner
- Tools: ArgoCD, FluxCD

- MLOps

- Focuses on deploying ML models in a continuous manner
- Tools: MLFlow

Extra notes

- Matrix tests/builds
 - Different python versions
 - Different OS
- Never put secrets in code
 - Pass them via env vars to CI/CD
 - Use a secret/config manager (AWS SSM, Hashicorp Vault, ...)
- Only build artifacts once (artifacts = container, package, ...)
 - Retag artifacts, instead of rebuilding
- Make your CI/CD pipeline as repeatable as possible
 - Hardcoding versions (preferably hash version) will save you a lot of pain
- Smoke tests
 - “Sanity” check after deploy/release

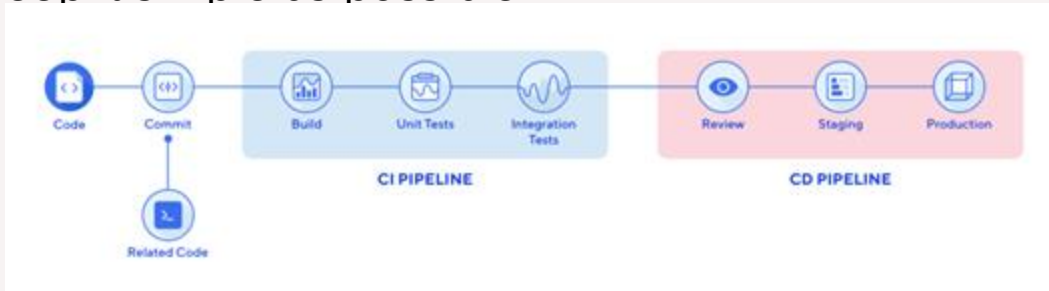
CI/CD Demo

CI/CD - Example

DEMO

Summary

- **DevOps**: mindset => end to end responsibility
- Getting your code from your PC to Production
 - Easy to collaborate with your team
 - As automated as possible
 - Keeping code quality high
- How you do it will vary from client to client
- Try to keep it simple as possible



CI/CD Exercises

Exercises

- Create CI/CD pipeline for **AWS Lambda (python) deployment**
- 3 exercises:
 - Exercise 1: Continuous Integration
 - Exercise 2: Continuous Delivery
 - **Bonus** Exercise 3: Continuous Deployment

Exercises - general info

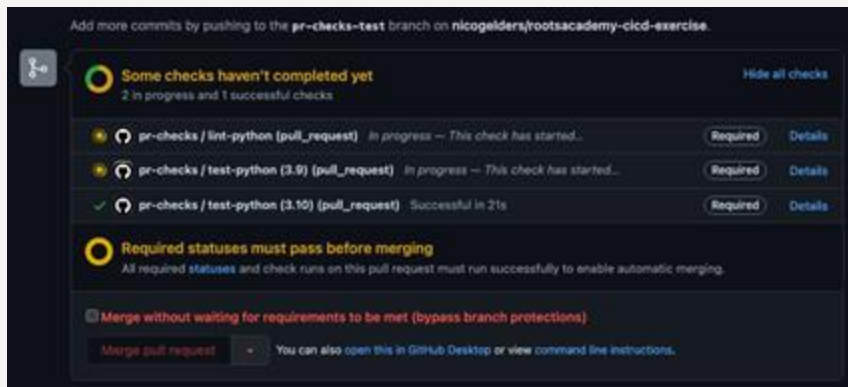
- Fork repo: <https://github.com/datarootsio/rootsacademy-cicd-exercise>
- Have a good look at the code before you start
- replace "<put-your-name-here>" in all the repo with your name
- Get a good look at GitHub actions documentation
- If you create a PR make sure you target your **own** fork
- Good starting point: <https://docs.github.com/en/actions/quickstart>
- Tip: Look around our github for CI/CD examples

Exercise 1 - Continuous Integration

Goal: assert quality of code

Requirements:

- CI pipeline:
 - Lint code: python + tf (use pre-commit)
 - test python: run pytest
- Pipeline runs on **every push to the repo**
- Only add code to main via PR merge (via github repo settings)
- Block PR if CI pipeline fails (via github repo settings)
- **Bonus:** try a testing matrix for python 3.9 and 3.10

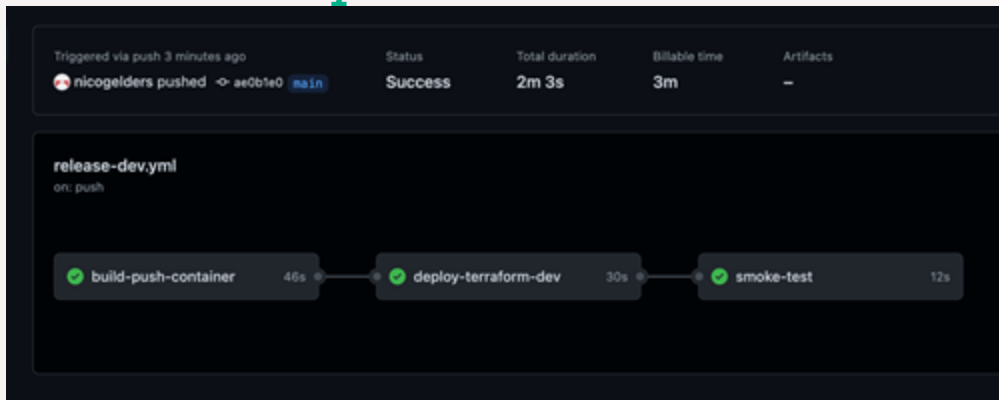


Exercise 2 - Continuous Delivery

Goal: deploy to dev

Requirements:

- CD pipeline
 - Docker container
 - Build
 - Push to ecr (With AWS creds; see AWS > IAM > Users > ccd-exercise-deploy-<name>)
 - Deploy to dev (with terraform)
 - AWS ECR repo name = rootsacademy/cicd-exercise-<student_name>
 - Tag should be: "<env>-<version-from-pyproject.toml>"
- Pipeline only runs when **push to main**
- **Bonus:** run smoke test for dev (don't smoke, bad for health)



Bonus Exercise 3 - Continuous Deployment

Goal: deploy to production

Requirements:

- CD pipeline
 - Retag image from dev
 - Deploy to production
- Pipeline **only runs when a new tag is created**
- **Bonus:** run smoke test for pro